Aurora Borealis Prediction

Dhrithi Kiran
dhrithikiran26@gmail.com

The Big Idea
The goal of this project is to predict the Kp index 3 hours before it happens, using solar wind data measured at the L1 point between the Sun and Earth. Once we have that predicted Kp number, we can translate it into a real-world aurora forecast — telling someone whether the northern lights will be visible from their location.

The analogy I used to understand this: a weather sensor reading at 5 AM is used to predict the temperature at 6 AM. Similarly, solar wind measured right now will determine what Earth's magnetic field looks like 3 hours from now. The solar wind hits L1 first, then takes 30–90 minutes to travel to Earth. The NASA OMNI dataset already accounts for this travel time, so the timestamps in OMNI represent when the solar wind actually hits Earth, not when it was measured at the satellite. That is why OMNI was chosen over raw satellite data — no manual time shifting needed.

The Kp index is a number from 0 to 9 that measures how disturbed Earth's magnetic field is. Low Kp means a calm field and aurora only near the poles. High Kp means a geomagnetic storm and aurora visible much further from the poles, potentially over the UK, central Europe, or even the southern USA during extreme events. Aurora happens because solar wind particles, guided by Earth's magnetic field, collide with atmospheric gases near the poles and make them glow. When Bz (the north-south component of the interplanetary magnetic field) turns strongly southward (negative), it opens Earth's magnetosphere like a door, letting in more particles and pushing the aurora oval further toward the equator.

Step 1 — Data Source and Raw Format

- The data came from NASA's OMNI dataset, downloaded as a plain text file covering January 2025 to January 2026. The text file had no headers and used fixed-width spacing. Each row represented one hour of measurements.
- The columns were: Year, Day of Year, Hour, IMF Bz in GSM coordinates (nT), Proton Temperature (K), Proton Density (particles per cm^3), Solar Wind Speed (km/s), Flow Pressure (nPa), and the Kp index multiplied by 10 and Kp. So you have ***10 columns*** total after preprocessing.
  Columns → Year, DOY (day of year), Hour, Bz_GSM_nT, Proton_Temp_K, Proton_Density_cm3, Solar_Wind_Speed_kms, Flow_Pressure_nPa, Kp_x10, and Kp.
- The reason Kp is stored multiplied by 10 in OMNI is because Kp can take values like 2.3 or 5.7, and storing it as an integer times 10 avoids decimal point issues in the raw text format. So the first thing done was *dividing Kp_x10 by 10 to get the actual Kp value back*.
- GSM coordinates (Geocentric Solar Magnetospheric) were used for Bz rather than GSE (Geocentric Solar Ecliptic) because GSM is aligned with Earth's magnetic field axis, which is what physically matters for geomagnetic storms. GSE is aligned with Earth's orbital plane and is less relevant for magnetospheric physics.

Note:- For the models you are using this is sufficient but not large.

1. Linear Regression works fine with this.
2. Random Forest with 200 trees is also fine and will generalise reasonably well.
3. The Neural Network (128, 64, 32) is the one most at risk because neural networks generally prefer tens of thousands of samples to train robustly. With only ~8,750 rows split 80/20, you have

roughly 7,000 training samples and 1,750 test samples. The MLP will train but expect underfit or overfitting.

Step 2 — Preprocessing

- The raw text file was loaded into Python using pandas with whitespace as the delimiter and no header row. Column names were manually assigned.
- The Kp column was created by dividing Kp_x10 by 10.
- Missing values were filled with the ***median*** of each column, which is more robust than using the mean because solar wind data can have *extreme outliers* that would distort a mean.
- Some extremely large outlier-looking values might have passed through as real data. Replace 9999, 99999, 999999, and 1e5 with NaN and then replaced with median.
- All numeric columns were explicitly cast to float to prevent any type mismatch issues later. The cleaned dataset was saved as a CSV file. A sample of the first 100 rows was also saved separately just for quick visual inspection.

- Kp in your data ranges from 0 to around 9, but most hours will be *between 1 and 4* because geomagnetic storms are relatively rare. The distribution is right-skewed — lots of quiet hours and few storm hours. This creates a *challenge for ML models* because they see far more examples of low Kp than high Kp during training. The models will therefore tend to ***predict low Kp more accurately and may underestimate storm peaks.***

Step 3 — Exploratory Data Analysis (EDA)*analyze the output!!!!!!!*

- Before building any models, the data was explored visually and statistically to understand its structure and patterns. A Datetime column was created by combining the Year, Day of Year, and Hour columns into a proper timestamp.
- The basic info includes checking the data types, check for unexpected nulls, and see the range and spread of each variable. Kp was then plotted over the full year as a time series to see when storms happened and how often. The solar wind parameters (Bz, temperature, density, speed, flow pressure) were each plotted over time as well.
- A correlation heatmap was produced to see which solar wind parameters correlate most strongly with Kp. Bz is expected to show the strongest relationship because of the physics explained above. Distribution histograms with KDE curves showed whether variables were normally distributed or skewed. Kp tends to be right-skewed because quiet periods (low Kp) are much more common than storms. Boxplots showed the spread and flagged extreme outlier values that might need cleaning.
- Scatter plots of each solar wind parameter against Kp visually confirmed which relationships exist and whether they are linear or nonlinear. Rolling averages of Kp at 24 hours and 72 hours were plotted to see trends and multi-day storm patterns. Finally, lagged versions of the solar wind parameters (1 hour, 2 hour, 3 hour, 6 hour delays) were plotted over time to visually inspect how much temporal offset exists between solar wind changes and Kp responses.

Step 4 — Feature Engineering

- Raw solar wind values at a single point in time are not enough to predict Kp well. Geomagnetic activity has memory — a storm that started 3 hours ago is still ongoing now. The feature engineering step created additional columns that capture this memory and temporal structure.
- Missing values were handled first using linear interpolation for short gaps (up to 3 hours), then forward fill and backward fill for any remaining gaps. This preserves the time-series continuity rather than creating sudden jumps.

- Lag features were created for Bz, solar wind speed, proton density, and Kp itself at delays of 1, 2, 3, and 6 hours. For example, Bz_lag_1 is Bz from one hour ago, Bz_lag_3 is Bz from three hours ago. These let the model see the recent history of each parameter, not just the current value. Past Kp values were particularly important because geomagnetic storms persist — if Kp was 6 two hours ago, it is likely still elevated now.
- Rolling statistics captured trends over short windows. Rolling means over 1, 3, and 6 hours gave a smoothed average of recent conditions. Rolling min and max captured the extremes within those windows. Rolling standard deviation over 3 and 6 hours captured how much the parameters were fluctuating (window of 1 was skipped for standard deviation because you need at least 2 values to compute it).
- Rate of change features (first differences) were computed for Bz, speed, and density. A sudden drop in Bz or a spike in speed is often the trigger for a storm onset, so capturing how fast things are changing adds important predictive signal.
- Physics-based interaction features were created to reflect known solar wind coupling mechanisms. Bz multiplied by speed captures the idea that southward IMF at high speed is more dangerous than southward IMF at low speed. Bz multiplied by density weighted the field by particle count. Speed squared multiplied by density is related to the solar wind dynamic pressure. An electric field proxy was computed as speed times the absolute value of Bz, and a southward-only version was computed that is zero when Bz is positive and only active when Bz is negative, directly encoding the physics of reconnection. An Alfvén Mach number proxy was also computed.
- Southward Bz features were created to capture how long Bz had been continuously negative. A binary flag was set to 1 whenever Bz was negative, and a running counter tracked the consecutive duration of each southward episode. A southward strength column gave the absolute magnitude of Bz only when it was negative.
- Time-based features — hour of day, day of year, and month — were added because geomagnetic activity has diurnal and seasonal patterns.
- After all this, the first 6 rows were dropped because those rows have NaN values in their lag features (there is no "1 hour ago" for the first row in the dataset). Rows with physically invalid solar wind speed or density values (zero or negative) were also removed.

- ***After adding lag features:*** dataset grew from 10 columns to 26 columns. The 16 new columns are the lag features. You created 4 lag steps (1, 2, 3, 6 hours) for 4 variables (Bz, solar wind speed, density, Kp)
  Each lag column shifts the original values down by that many rows. For example, Bz_lag_1 in row 10 contains the Bz value from row 9. This means the first 6 rows will have NaN in their lag_6 columns because there is no data 6 hours before the very first row. This is expected and handled later in cleanup.
- The ***rolling statistics*** added 27 new columns (9 mean columns, 9 min/max columns, and 6 std columns, but the exact count depends on how windows were applied across 3 variables — the result brings you to 53 columns). These capture the average, minimum, maximum, and variability of Bz, speed, and density over the last 1, 3, and 6 hours. ***Rate of change*** added 3 columns (dBz_dt, dV_dt, dD_dt). ***Physics features*** added 6 columns: Bz_times_V, Bz_times_D, V2_times_D, Electric_Field_Proxy, Electric_Field_South, and Alfven_Mach_Proxy. ***Southward Bz features*** added 3 columns: Bz_negative (0 or 1 flag), Bz_neg_duration (how many consecutive hours Bz has been southward), and Bz_southward_strength (the magnitude of Bz only when it is negative, zero otherwise)

Step 5 — Creating the Target and Splitting the Data

- A timestamp column was constructed by combining the Year, DOY (day of year), and Hour columns into a single datetime object, allowing the data to be treated as a proper time series.
- The dataset was sorted chronologically before any splitting — this is critical because leaving data unsorted would mean the train and test sets could contain randomly interleaved time points, which would constitute data leakage.
- The target variable `Kp_3h_ahead` was created by shifting the Kp column backward by 3 positions, so each row's target reflects what the Kp index will be three hours into the future. The last 3 rows were dropped because they have no valid future value to predict.
- The dataset was split 80/20 using index position rather than random sampling. The first 80% of chronological rows (7,144 samples, January 2025 to late October 2025) became the training set, and the remaining 20% (1,786 samples, late October 2025 to January 2026) became the test set. This preserves temporal order and prevents the model from ever seeing future data during training.
- Both splits were saved to CSV files for use by all subsequent model notebooks.

Step 6 — Model Training

- Three models were trained, each increasing in complexity.

1. ***Linear Regression*** was the baseline. It is the simplest possible approach and gives reference pt
   It assumes that Kp = weighted sum of the input features.
   It is also the most interpretable model because you can look at the coefficient for each feature and understand its direction and magnitude of influence.
   For example, a large negative coefficient on Bz_lag_1 would confirm that more negative Bz predicts higher Kp, which matches the physics.
   Linear Regression cannot handle NaN values. The *imputer* was fit only on training data and then applied to the test data using transform (not fit_transform), which is important — if you fit the imputer on test data you are allowing test information to influence preprocessing, which is another form of leakage.

- The train and test CSVs were loaded with `parse_dates` on the Timestamp column. The training set contained 7,144 rows and the test set 1,786 rows, each with 68 columns.
- The target column (`Kp_3h_ahead`) and the `Timestamp` column were excluded from the feature matrix. The Timestamp is excluded because the time information is already represented numerically through engineered features such as `Hour_of_Day`, `Day_of_Year`, and `Month`. This left 66 feature columns. After further filtering to only numeric types, 65 features were used for training.
- A `SimpleImputer` with median strategy was fitted exclusively on the training data and then applied to both train and test sets. Fitting only on training data is important — using the test set in the imputer fit would leak information about the test distribution into the model.
- A standard `LinearRegression` model from scikit-learn was trained. It learned 65 coefficients (one weight per feature) plus one bias term, representing the model: $Kp = w_1 \cdot feature_1 + w_2 \cdot feature_2 + \ldots + bias$.
- Evaluation on the test set produced **MAE: 0.6426, RMSE: 0.8492, R^2: 0.6498**. This serves as the baseline — any more complex model should be expected to improve on these numbers.

2. **Random Forest** was the second model.
   It builds 200 decision trees, each trained on a random subset of the data and features. Each tree independently makes a prediction and the final output is the average across all trees.
   This ensemble approach makes the model more stable and less prone to overfitting
   Random Forest can naturally capture nonlinear relationships and interactions between features

for example, it can learn that Kp spikes when Bz is negative AND speed is high simultaneously, which is something a linear model cannot do on its own. It also handles the engineered interaction terms and rolling features well. max_depth was set to 20 to prevent the trees from growing so deep that they memorise the training data.

- The same 65 numeric features used in linear regression were selected, excluding the target, Timestamp, and Datetime columns. No separate imputation step was applied here because Random Forest in scikit-learn can handle missing values internally in this configuration, though the dataset was already largely clean from earlier preprocessing.
- A `RandomForestRegressor` was configured with 200 trees (`n_estimators=200`), a maximum tree depth of 20 (`max_depth=20`), and `n_jobs=-1` to use all available CPU cores in parallel. The random state was fixed at 42 for reproducibility.
- Evaluating on the training set gave **MAE: 0.1774, RMSE: 0.2305, R^2: 0.9702**, while the test set gave **MAE: 0.6668, RMSE: 0.8829, R^2: 0.6215**. The large gap between training and test performance is a sign of overfitting — the model memorised the training data very well but did not generalise as strongly to unseen data. Despite this, its test MAE of 0.6668 is only marginally worse than linear regression's 0.6426, suggesting the added complexity of the forest did not deliver a meaningful improvement on this dataset in its current configuration.

3. **Neural Network (MLP)**
   Using scikit-learn's MLPRegressor with three hidden layers of 128, 64, and 32 neurons using ReLU activation and the Adam optimizer.
   Neural networks learn by adjusting internal weights to minimise the difference between predicted and actual Kp across many passes through the training data.
   Feature scaling using StandardScaler was essential before training because neural networks are sensitive to the scale of input values — a feature measured in thousands (like temperature in Kelvin) would completely dominate over a feature measured in single digits (like Kp) without scaling. The scaler was again fit only on training data and applied to test data.

- The same 65-feature matrix was used. Before training, two preprocessing steps were applied that are strictly required for neural networks: first, a median `SimpleImputer` was fitted on the training set and applied to both sets to remove any remaining NaN values; second, a `StandardScaler` was fitted on the training set and applied to both sets to normalise all features to zero mean and unit variance. Neural networks are sensitive to feature scale — unscaled inputs cause some neurons to receive very large activations while others receive near-zero values, which destabilises gradient-based optimisation. The scaler was saved alongside the model because the exact same transformation must be applied to any new data at inference time.
- The network architecture was a `MLPRegressor` with three hidden layers of sizes 128, 64, and 32 neurons respectively, using ReLU activation and the Adam optimiser. The maximum number of training iterations was set to 200.
- A convergence warning was raised, indicating that the optimiser had not fully converged within 200 iterations — the model was still improving when training was stopped. This means the results below represent an undertrained network and performance could improve by increasing `max_iter`.
- Despite fitting the training data reasonably well (**MAE: 0.2264, R^2: 0.9511**), the test set performance was substantially worse: **MAE: 0.9818, RMSE: 1.3577, R^2: 0.1048**. An R^2 of 0.10 means the model explains only about 10% of the variance in the test set, which is worse than both linear regression and random forest. This is a combination of the early stopping due to non-convergence and likely overfitting, since neural networks generally require more careful regularisation (dropout, weight decay) and hyperparameter tuning to generalise well on tabular data of this size.

Step 7 — Evaluation

- Each model was evaluated on both the training set and the test set using three metrics. MAE (Mean Absolute Error) gives the average prediction error in Kp units — easy to interpret. RMSE (Root Mean Squared Error) gives a similar measure but penalises large errors more heavily, which matters in space weather because missing a big storm is much worse than being slightly off during a quiet period. R^2 tells you what fraction of the variance in Kp the model explains — 1.0 is perfect, 0.0 means the model is no better than always predicting the mean Kp.
- Comparing train and test metrics tells you whether a model has overfit. If train R^2 is 0.95 but test R^2 is 0.50, the model has essentially memorised the training data and not learned general patterns.
- Linear Regression is least likely to overfit. Random Forest can overfit if trees are too deep. Neural Networks are most prone to overfitting, especially with limited data.

- Each model was evaluated on both the training set and the test set using three metrics. MAE (Mean Absolute Error) gives the average prediction error in Kp units and is the most intuitive — an MAE of 0.64 means the model's predictions are off by 0.64 Kp units on average. RMSE (Root Mean Squared Error) penalises large errors more heavily than MAE, which matters in space weather because missing a large storm is far more consequential than being slightly off during a quiet period. R^2 tells you what fraction of the variance in Kp the model explains — 1.0 is perfect, 0.0 means the model does no better than always predicting the mean Kp.
- Comparing train and test metrics is how you diagnose overfitting. A model that scores very high on training but much lower on the test set has memorised the training data rather than learning general patterns that transfer to new data.

Model	Train MAE	Test MAE	Train R^2	Test R^2
Linear Regression	—	0.6426	—	0.6498
Random Forest	0.1774	0.6668	0.9702	0.6215
Neural Network	0.2264	0.9818	0.9511	0.1048

- Linear regression produced the best-balanced result — no dramatic gap between train and test, and a test R^2 of 0.65 meaning it explains roughly 65% of the variance in Kp on unseen data. As the simplest model it is also least prone to overfitting.
- Random Forest showed a very large train–test gap (R^2 of 0.97 vs 0.62), which is a clear sign of overfitting. The trees memorised fine-grained patterns in the training period that did not persist into the test period. Despite having 200 trees and a depth cap of 20, the model did not meaningfully outperform linear regression on the test set, suggesting the additional complexity was not justified here.
- The neural network performed worst overall on the test set, with an R^2 of just 0.10 — meaning it explains almost none of the variance in unseen Kp values. This was compounded by the convergence warning raised during training, indicating the Adam optimiser had not yet reached a stable solution within the 200-iteration limit. The combination of non-convergence, overfitting, and the lack of regularisation techniques such as dropout or weight decay explains the poor generalisation.

Step 8 — Aurora Interpretation

- The models output a raw Kp number. Each Kp integer from 0 to 9 corresponds to a NOAA G-scale storm rating (G0 through G5), an equatorward boundary latitude where aurora becomes visible, and a description of where in the world aurora can be seen.
  For example Kp 4 means aurora might reach northern Scotland. Kp 6 means a G2 storm and aurora visible from southern England and central Europe. Kp 9 is an extreme G5 storm and aurora visible from the Mediterranean.
- The idea is that after the model predicts Kp = 5.7 for 3 hours from now, we round that to 6, look it up in the table, and produce a human-readable forecast. This is the bridge between the raw machine learning output and the real-world question the project was built to answer: will the aurora be visible tonight?

- The raw model output is a continuous Kp number. To make this useful for a real forecast, the predicted value is rounded to the nearest integer and mapped against the NOAA G-scale storm classification table, which links each Kp level to a geomagnetic storm severity, an approximate equatorward visibility boundary, and a plain-language description of where aurora is likely to be seen.

Predicted Kp	**NOAA G-Scale**	**Aurora Visibility**
0 – 2	G0 (No storm)	Polar regions only (northern Norway, Iceland)
3	G0	Northern Scandinavia, northern Canada
4	G0	Northern Scotland, northern Russia
5	G1 (Minor)	Scotland, southern Scandinavia
6	G2 (Moderate)	Southern England, northern Germany, northern USA
7	G3 (Strong)	Central Europe, mid-latitude USA
8	G4 (Severe)	Southern Europe, southern USA
9	G5 (Extreme)	Mediterranean, northern Mexico — rare, historic storm level

- For example, if the model predicts Kp = 5.7 for three hours from now, that rounds to 6, maps to a G2 storm, and translates into a forecast that aurora should be visible from southern England and northern Germany under clear skies. This is the bridge between the raw machine learning output and the practical question the project was built to answer: will the aurora be visible tonight, and from where?

- The 3-hour prediction horizon was chosen deliberately to match the cadence at which Kp is officially reported and to give enough lead time for a person to make an observing decision before the window opens.